DRS MC508 – SS 2025
Lab Report Exercises 1–8

Simone Bianca Feldbacher

March 14, 2026

Abstract

The Course Distributed Real Time Systems focuses on the principles and technologies for building distributed real-time systems. The course covers fundamental topics like worst-case execution time analysis, real-time communication, scheduling algorithms, resources, and comunication. These concepts are applied using QNX 8 on a raspberry Pi 4. This report summarizes the 8 assignments completed during the course, highlighting the key concepts and techniques learned in each chapter.

Contents

1 Assignment 1 — WCET **4**
1.1 Assignment . . . 4
1.2 Implementation . . . 4
1.2.1 Initialization . . . 5
1.2.2 Data Acquisition . . . 5
1.2.3 Calculation . . . 5
1.2.4 State Transition . . . 6
1.2.5 Challenges . . . 7
1.3 Evaluation . . . 7
1.3.1 WCET analysis from the IAR Simulator Results . . . 7
1.3.2 WCET results from BoundT . . . 8
1.3.3 Discussion . . . 8

2 Assignment 2 — Scheduling policies **9**
2.1 Assignment . . . 9
2.2 Approach . . . 10
2.2.1 Cyclic Executive . . . 10
2.2.2 Round Robin . . . 10
2.2.3 Rate Monotonic . . . 10
2.3 Evaluation . . . 11

3 Assignment 3 - WCET calculator **13**
3.1 Assignment . . . 13
3.2 Implementation . . . 13
3.3 Evaluation . . . 14

4 Assignment 4 — Real-time Application with QNX, Profiling **15**
4.1 Assignment . . . 15
4.2 Approach . . . 15
4.2.1 Setup and Hello World . . . 16
4.2.2 In class task - measurement with logic analyzer . . . 16
4.2.3 Task 1: Airbag System . . . 16
4.2.4 Task 2: CRC16 Checksum . . . 17
4.2.5 Task 3: Diagnostic Task . . . 18
4.3 Results . . . 19
4.3.1 Shared Resources and Priority Inversion . . . 19

4.3.2 Alternative Resource Protection Methods 20

5 Assignment 5 — Real-time Application with QNX, HAM 21

6 Assignment 6 — Real-time Application with QNX, IPC Messaging 22

7 Assignment 7 — Real-time Application with QNX, Device Drivers 23

8 Assignment 8 — Real-time Application with QNX, CAN Bus 24

Chapter 1

Assignment 1 — WCET

1.1 Assignment

After learning about scheduling and worst-case execution time (WCET) analysis, the first assignment focuses on developing a simple real-time application and analyzing its scheduling behavior and WCET. A passenger protection system is a system that has strong requirements for real time and fault tolerance. The following tasks should be completed:

- Implement a simplified version of the Airbag Deployment Algorithm described in [1]. Using either C or C++, Don't use floating point numbers or OO features.
- Analyze the WCET using IAR embedded workbench.
- Use BoundT to analyze the WCET. compare both results.

1.2 Implementation

The provided paper and lecture slides provide a starting point and outline how to implement the airbag deployment algorithm. It works by consistently calculating the current change in velocity as well as the length of the acceleration graph. Based in these values the system enters a STANDBY state and fires the airbag if the right conditions are met. Three thresholds are defined:

- ΔV_1: The change in velocity must be greater than ΔV_1 to enter the STANDBY state.
- ΔV_2: If change in velocity exceeds this value the airbag is fired immediately. ΔV_2 must be greater than ΔV_1.
- L: The length of the acceleration graph must be greater than L during standby state to fire the airbag.

Before implementation the algorithm was broken down into steps and states as follows:

1.2.1 Initialization

Sliding windows are initialized as arrays of size N and filled with zeros. The system starts in the *NORMAL* state. Everything after this is executed in an endless loop. Rudimentary timing is implemented using `usleep(1000000 / SAMPLES_PER_SECOND)`.

```
    // Initialize arrays for storing time samples
    int t1[T_SIZE1] = {0}; // Array for first time series
    int t2[T_SIZE2] = {0}; // Array for second time series

    // Define thresholds for change in velocity
    int deltaV1 = 20; // Threshold for entering standby state
    int deltaV2 = 50; // Threshold for firing the airbag
    int LThreshold = 300; // Threshold for length of acceleration graph

    // Variables for current calculations
    int deltaV = 0; // Current change in velocity
    int L = 0; // Current length of acceleration graph

    // Variable to hold the current sample index
    int currentSample;

    // State of the airbag system
    AirbagState state = NORMAL; // Initial state is NORMAL
```

1.2.2 Data Acquisition

Query new data, random values in this case, and update the sliding windows by shifting existing values and adding the new data. This step happens regardless of state.

```
    static void processSample(int t1[], int size1, int t2[], int size2, int *deltaV,
     int *L)
    {
    int currentSample = rand() % (10 + 1 + 30) - 30;
    printf("Current Sample: %d\n", currentSample);

    slideWindow(t1, size1, currentSample);
    slideWindow(t2, size2, currentSample);

    *deltaV = determineDeltaV(t1, size1);
    *L = determineL(t2, size2);

    printf("DeltaV: %d, L: %d\n", *deltaV, *L);
}
```

1.2.3 Calculation

Calculate ΔV and L within the scope of the sliding windows, using the formulas presented in the paper.

```
    static int determineDeltaV(int window[], int size)
    {
    double deltaV = 0;

    for (int i = 0; i < size; i++)
```

```
    {
        deltaV += window[i] * 1.0 / SAMPLES_PER_SECOND;
    }
    return round(deltaV);
}

static int determineL(int window[], int size)
    {

    double l = 0, dy = 0;
    double dx = 1.0 / SAMPLES_PER_SECOND;
    for (int i = 0; i < size - 1; i++)
    {
        dy = window[i + 1] - window[i];
        l += sqrt(pow(dy, 2) + pow(dx, 2));
    }
    return (int)l;
}
```

1.2.4 State Transition

Possible states are *NORMAL*, *STANDBY* and *FIRE*. If ΔV exceeds ΔV_1 the system enters the standby state. If ΔV exceeds ΔV_2 the system fires immediately. If the system is in STANDBY state and L exceeds the threshold L, the system fires. If the system is in STANDBY state and ΔV drops below ΔV_1 the system goes back to normal. For clarity these are implemented as switch cases.

Possible transitions are:

- NORMAL → STANDBY → NORMAL
- NORMAL → STANDBY → FIRE

```
        case NORMAL:
            processSample(t1, T_SIZE1, t2, T_SIZE2, &deltaV, &L);
            if (abs(deltaV) > deltaV1)
            {
                state = STANDBY;
            }
            break;

        case STANDBY:
            processSample(t1, T_SIZE1, t2, T_SIZE2, &deltaV, &L);
            if (abs(deltaV) > deltaV2 || L > LThreshold)
            {
                state = FIRE;
            }
            if (abs(deltaV) < deltaV1)
            {
                state = NORMAL;
            }
            break;

        case FIRE:
            exit(0);
            break;
```

1.2.5 Challenges

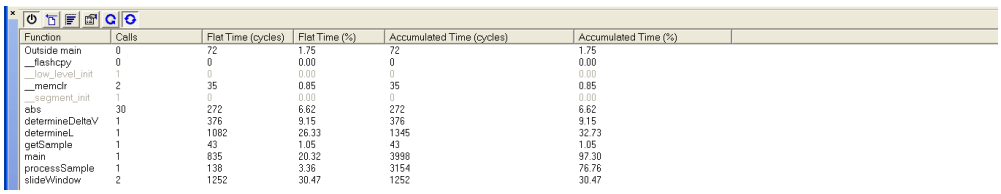
While the algorithm was simple to implement, trying to run it in the provided version of IAR Embedded Workbench caused some problems. Headers like time.h were not included per default and would have taken up too much space to compile in the free version. std:out was initially replaced with `printf` and later entirely omitted due to the same space concerns. Some rewrites were necessary to fix these problems.These problems were tackled as follows:

- unsupported headers: Removed time.h since only the WCET of the actual code execution matters for WCET,
- Error e89: Caused by exceeding the 4KB object code limit of the kick start version. Remove the studio.h header to save on object code. This also means removing all `printf` statements, which did not matter for the WCET analysis anyway.
- Error e16: Occurred during building. Was fixed by selecting the *ATmega328P* as the target processor.

BoundT presented it's own set of challenges. It would, despite my best efforts, not compile on my machine so the pre-installed version in the proved VM was used instead. The 1024kb limit was no problem. The require .elf file was compiled from VS code using gcc, which failed due to missing STABS symbols. Using an older version of the compiler and including stabs finally lead to a usable result.

1.3 Evaluation

After getting *Wayland* and *VMware Workstation* to stop fighting using the true and trust method of copying a fix from Reddit, all the analysis was done on the provided version of Windows XP.



Function	Calls	Flat Time (cycles)	Flat Time (%)	Accumulated Time (cycles)	Accumulated Time (%)
Outside main	0	72	1.75	72	1.75
__flashcpy	0	0	0.00	0	0.00
__low_level_init	1	0	0.00	0	0.00
__memclr	2	35	0.85	35	0.85
__segment_init	1	0	0.00	0	0.00
abs	30	272	6.62	272	6.62
determineDeltaV	1	376	9.15	376	9.15
determineL	1	1082	26.33	1345	32.73
getSample	1	43	1.05	43	1.05
main	1	835	20.32	3998	97.30
processSample	1	138	3.36	3154	76.76
slideWindow	2	1252	30.47	1252	30.47

Figure 1.1: Profiling view from IAR Embedded Workbench Simulator.

1.3.1 WCET analysis from the IAR Simulator Results

Debugging the code in IAR resulted in the following execution times for one loop after the initial setup (seen in 1.1) for the function implemented. `slideWindow` gets called twice per loop because the sliding windows are implemented as two separate arrays. The Accumulated Time for `main` shows that it takes 3998 cycles to execute one loop. At a clock speed of 16MHz this would take 0.25ms to execute, well under the 30ms requirement for the airbag system. Such a short time is expected because the sensor reading is not accounted for and the sliding windows are small with only a length of 20

and 30 respectively. Increasing both by 10 already increases the time to 5300 cycles or 0.33ms.

1.3.2 WCET results from BoundT

After compiling the algorithm for the Atmega128 (the Atmega328P was not supported in BoundT) the analysis was run with the following parameters:

```
boundt_avr -device=ATmega128 -elf_sym -max_steps 1024 -show times
airbag_atmega128_stabs.elf main
```

The resulting WCET in CPU cycles were:

- 2386 cycles for the `processSample` function, which coposes fetching the samples, sliding the windows and calculating ΔV and L.
- 3144 cycles for the `main` function, which includes the state transitions and the calls to `processSample`.

These results are lower than the ones obtained from the IAR Simulator. This difference could come from the fact that the code was compiled for a different processor, or that the compilation in VS code performed some optimizations that the IAR compiler did not.

1.3.3 Discussion

For both results it is important to keep in mind that the WCET from this example is not realistic since the code does not include the time it takes to read the sensors, which is a crucial part of the airbag system. The results are only for the processing of a small set of sample data and the state transitions, to satisfy the goal of this assignment which was to get familiar with the tools and the process of WCET analysis.

Chapter 2

Assignment 2 — Scheduling policies

2.1 Assignment

After having determined the WCET for one of the tasks in the airbag system to be about 0.25ms or 250μs in the last assignment, now we have to consider other tasks that get executed periodically in the system as well. From the lecture slides the following three additional tasks were selected: The goal of assignment 2 is to work with three

Table 2.1: Tasks with their periods and worst-case execution times

Task	Description	Period	WCET
τ_1	As determined in the last chapter. This task is responsible for the airbag deployment.	1ms	0.25ms
τ_2	Responsible for gathering data from an additional satellite accelerometer.	1ms	0.1ms
τ_3	Responsible for verifying the decision of the airbag system.	1ms	0.01ms
τ_4	Responsible for communicating with a Diagnostic interface.	100ms	5ms

different scheduling policies and use *Cheddar* to simulate the resulting schedules. For the different policies the following variables should be determined:

- **Rate Monotonic (RM)**: A priority based strategy that assumes all tasks are periodic with their deadline being this period. Tasks with shorter periods get higher priorities. Perform tests in Cheddar to determine if the system is schedulable under this policy. Perform the Liu/Layland and hyperbolic tests.
- **Round Robin (RR)**: Fair policy that lets tasks execute until they give up control to the scheduler. Determine the necessary time quantum and how many context switches are necessary. Use Cheddar to simulate the resulting schedule.
- **Cyclic Executive (CE)**: Fair policy that runs all tasks to completion in the same order repeatedly. Determine the minor and major cycle and if a feasible schedule

is possible.

2.2 Approach

The exercises were approached in order of complexity, starting with cyclic executive.

2.2.1 Cyclic Executive

The greatest common divisor of the tasks from 2.1 is one 1ms, which determines the minor cycle. The major cycle, the least common multiple of the task periods is 100ms. While τ_1, τ_2 and τ_3, who get executed every minor cycle, are feasible to execute within one minor cycle since their execution times only add up to 0.36ms, τ_4 cannot be executed within the minor cycle. Its execution time of 5ms alone is greater than one minor cycle, meaning that during its execution the other tasks cannot meet their deadlines. Therefore, a feasible schedule is not possible under the cyclic executive policy.

2.2.2 Round Robin

Since τ_4 takes so long to execute, it must be interrupted so the other tasks get the chance to meet their deadlines. The first three tasks need to be executed every 1ms, but they also need time to execute, so the quantum needs to be smaller than that. A quantum of 0.6 ms would interrupt τ_4 with 0.4 ms left until the deadline of the first three tasks, which is enough for all three of them to execute. To verify this, a simulation was run in Cheddar with the following parameters 2.2.2.

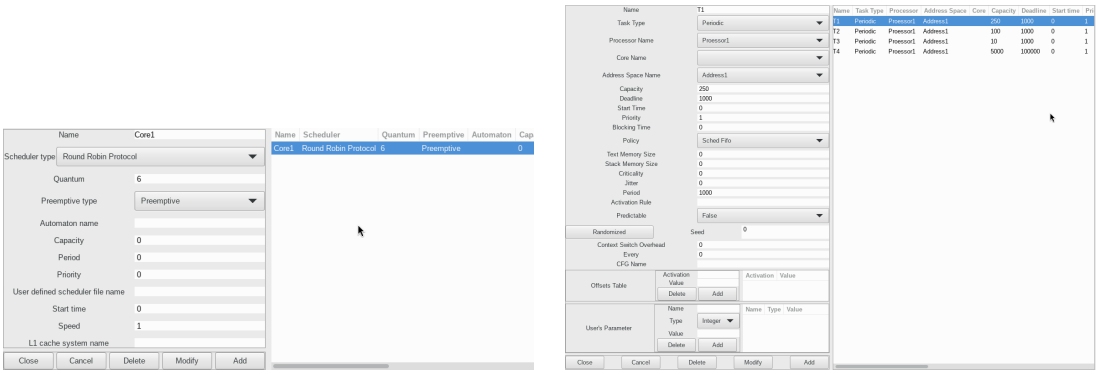


Figure 2.1: Parameters for simulating Round Robin scheduling in Cheddar. The left image shows the core, the right image shows the tasks and their parameters.

The simulation confirms that this schedule is feasible but also requires over 200 context switches per 100ms. The high number of context switches is a consequence of the periods and execution time of the tasks varying so greatly.

2.2.3 Rate Monotonic

In Rate Monotonic scheduling, the tasks with the shortest period get the highest priority. In this case, τ_1, τ_2 and τ_3 would have higher priority than τ_4.

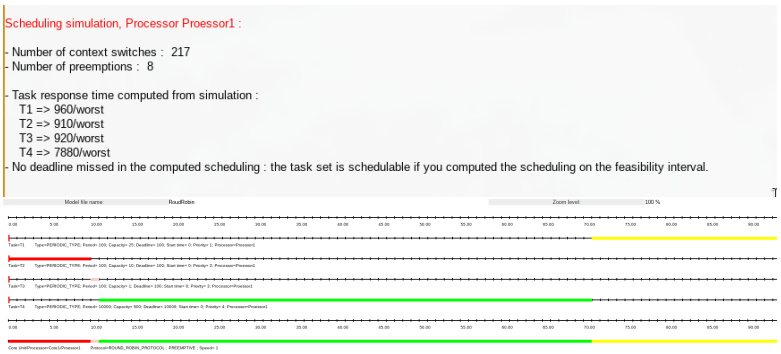


Figure 2.2: Output from Cheddar for simulating Round Robin scheduling with the parameters from Table 2.1 and a quantum of 0.6ms. Numbers in the top image are in microseconds over the span of 100ms, or one period of task 4. Numbers in the bottom image are multiplied by 100 since Cheddar does not allow for decimal inputs.

The schedulability condition by Liu and Layland is

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n\left(2^{1/n} - 1\right),$$

where C_i is the (worst-case) execution time of task i, T_i is the period of task i, and n is the total number of tasks.

For the tasks from Table 2.1 the utilization U results in

$$U = 0.41 \leq 0,76$$

which means that the system is schedulable under the Rate Monotonic policy.

The hyperbolic bound test is given by this condition:

$$U = \prod_{i=1}^{n} \left(\frac{c_i}{T_i} + 1\right) \leq 2$$

which resolves to

$$U = 1.45 \leq 2$$

meaning this test passes as well.

To run the simulation in Cheddar, only the schedule type parameter in the core needs to be changed in the setup from the Round Robin simulation. Custom task priorities did not affect the resulting schedule, which is expected since the priorities are determined by the periods of the tasks. Figure 2.3 shows the simulation results.

2.3 Evaluation

Table 2.2 shows the simulation results from Cheddar for both scheduling policies over a 100ms interval. Rate Monotonic scheduling surprisingly results in more context switches than Round Robin, yet it also decreases the WCET for the first three tasks significantly.

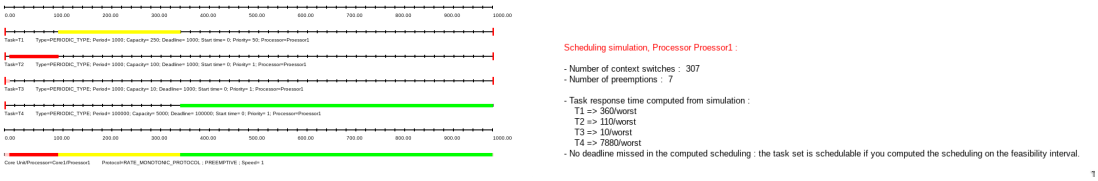


Figure 2.3: Rate Monotonic scheduling simulation in Cheddar showing the timeline (left) and simulation statistics (right).

This is because τ_4 gets preempted more often, which allows the other tasks to execute sooner. The number of preemptions is similar for both policies, which is expected since τ_4 needs to be preempted at least 7 times to allow the other tasks to execute.

Table 2.2: Comparison of Round Robin and Rate Monotonic scheduling simulation results

Metric	**Round Robin**	**Rate Monotonic**
Context switches	217	307
Preemptions	8	7
τ_1 response time (worst)	960 μs	360 μs
τ_2 response time (worst)	910 μs	110 μs
τ_3 response time (worst)	920 μs	10 μs
τ_4 response time (worst)	7880 μs	7880 μs
Deadlines missed	0	0
Schedulable	Yes	Yes

Chapter 3

Assignment 3 - WCET calculator

3.1 Assignment

After learning about various scheduling policies and blocking tasks, the assignment for Lab 3 focuses on writing a short program that calculates worst case response time using *Deadline Monotonic Scheduling*.

3.2 Implementation

The assignemnt was completed using C#. Input data was read froma csv file using the *csvHelper* library. The input includes *Task ID*, *Period*, *Deadline*, *Execution Time* and *Priority* as shown in Table 3.2.

Task	**Period**	**Deadline**	**Execution Time**	**Priority**
1	250	10	5	1
2	10	10	2	2
3	330	50	25	3
4	1000	1000	29	4

Table 3.1: Task Parameters

The algorithm tasks the task list and TaskID to calculate the time for as input and uses the formula 3.2 to calculate the worst case response time. The algorithm iteratively calculates the response time until it converges or exceeds the deadline.

$$R_i^{n+1} = C_i + \sum_{K \in hp(i)} \left\lceil \frac{R_i^n}{T_K} \right\rceil C_K$$

Algorithm: Worst-Case Response Time (WCET)

Input: Task set $\mathcal{T}$, task τ_k
Output: Worst-case response time R_k

1. Select task $\tau_k \in \mathcal{T}$
2. $\mathcal{HP} \leftarrow \{\tau_i \in \mathcal{T} \mid \text{priority}(\tau_i) > \text{priority}(\tau_k)\}$
3. $R_{\text{prev}} \leftarrow -1$
4. $R \leftarrow 0$
5. **while** $R \neq R_{\text{prev}}$ **do**
   5.1 $R_{\text{prev}} \leftarrow R$
   5.2 $R \leftarrow C_k$
   5.3 **for each** $\tau_i \in \mathcal{HP}$ **do**
   $R \leftarrow R + \left\lceil \frac{R_{\text{prev}}}{T_i} \right\rceil C_i$
   5.4 **end for**
6. **end while**
7. **return** R

3.3 Evaluation

To veryfiy the implementation, the same input data as shown in the slides was used. Calculating the response time for Task 3 returns 38ms which machtes the result shown on the slides. The response times for the other tasks are shown below:

Task	Response Time
1	5
2	7
3	38
4	75

Table 3.2: Resulting Response Times

As expected the Response time is higher for tasks with a higher priority.

Chapter 4

Assignment 4 — Real-time Application with QNX, Profiling

4.1 Assignment

After covering RTOSs and their components in detail in the lecture and learning how to schedule periodic tasks on QNX, this assignment focuses on getting started developing for QNX on the Raspberry Pi 4. After getting familiar with the hardware setup and remote debugging, the goal is to create 2 periodic tasks using the POSIX API and monitor their execution using the system profiler and an external logic analyzer. Code for this implementation as well as those of the following tasks can be found in the GitHub repository for this course [1].

4.2 Approach

Since the assignment is focused on working with threads rather than the precise code inside of them, simplified or dummy algorithms were used for some of the tasks. `clock_nanosleep` was used to achieve the periodic execution of the tasks.

```
struct timespec next;
clock_gettime(CLOCK_MONOTONIC, &next);
for(;;)
{
  // Sample Accelerometer Data

  // Advance deadline by period

  next.tv_nsec += 1000000; // 1 ms
  if(next.tv_nsec >= 1000000000)
  {
    next.tv_sec += 1;
    next.tv_nsec -= 1000000000;
  }
  clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &next, NULL);
}
```

[1] https://github.com/feldbacherSimone/Assignment_04

4.2.1 Setup and Hello World

The first step was to acquire the necessary education license for QNX 8.3 and set up the required extension for Visual Studio. The QNX image was flashed onto an SD card and run on the Raspberry Pi 4. To allow the device to be connected over ethernet the IP address of the Pi was statically assigned as 192.168.1.100, matching the configuration of the image, and configured in the IDE. With the setup complete the first "Hello World" application was created and successfully deployed to the Raspberry Pi. The setup and successful first execution are shown in Figure 4.1.

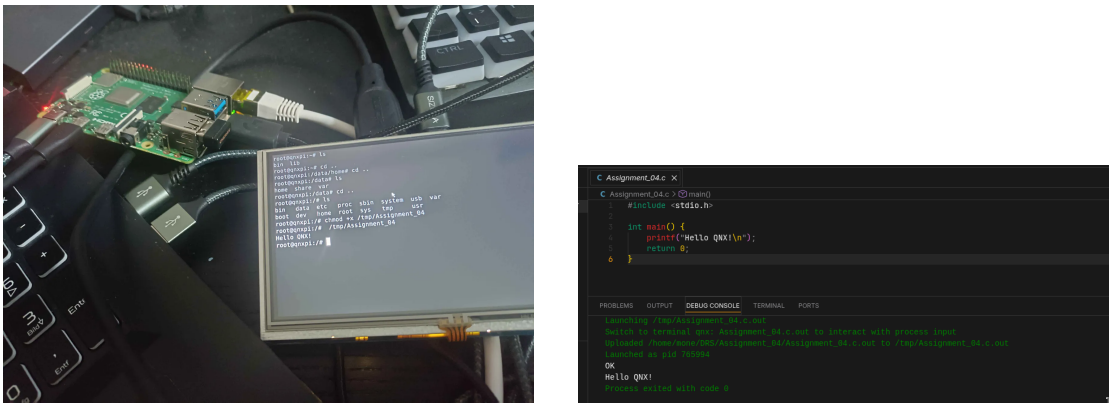


Figure 4.1: QNX setup on Raspberry Pi 4 (left) and successful Hello QNX execution (right).

4.2.2 In class task - measurement with logic analyzer

During the lecture a simple task was set up to toggle one of the GPIO pins on the Raspberry Pi. Using a logic analyzer the timing of the toggling was measured and compared to the expected timing. The results are shown in Figure 4.2. Surprising was that even when using a tiny delay (few nanoseconds) the period would increase to almost 2ms. This was caused by the minimum timer resolution on the QNX image being 1ms. Any delay is rounded up to the next multiple of 1ms, which is why the period increased to almost 2ms.

4.2.3 Task 1: Airbag System

In the first iteration the airbag thread only printed random numbers, which ended up being so simple that the execution time did not show up in the traces. Therefore, the full algorithm from exercise 1 was ported over with increased array lengths to accumulate a more realistic processing time, using the system profiler provided by VS Code. For the RPi4, the execution times shown in Figure 4.3 were measured with sliding windows of 6000 and 8000 samples. The trace kept failing on short periods, so in the example the period was set to 1s, since it was the execution time that mattered.

After some spontaneous, seemingly unrelated failure of my hardware, I switched to using a virtual QNX target running in QEMU. Here the trace was successful even with

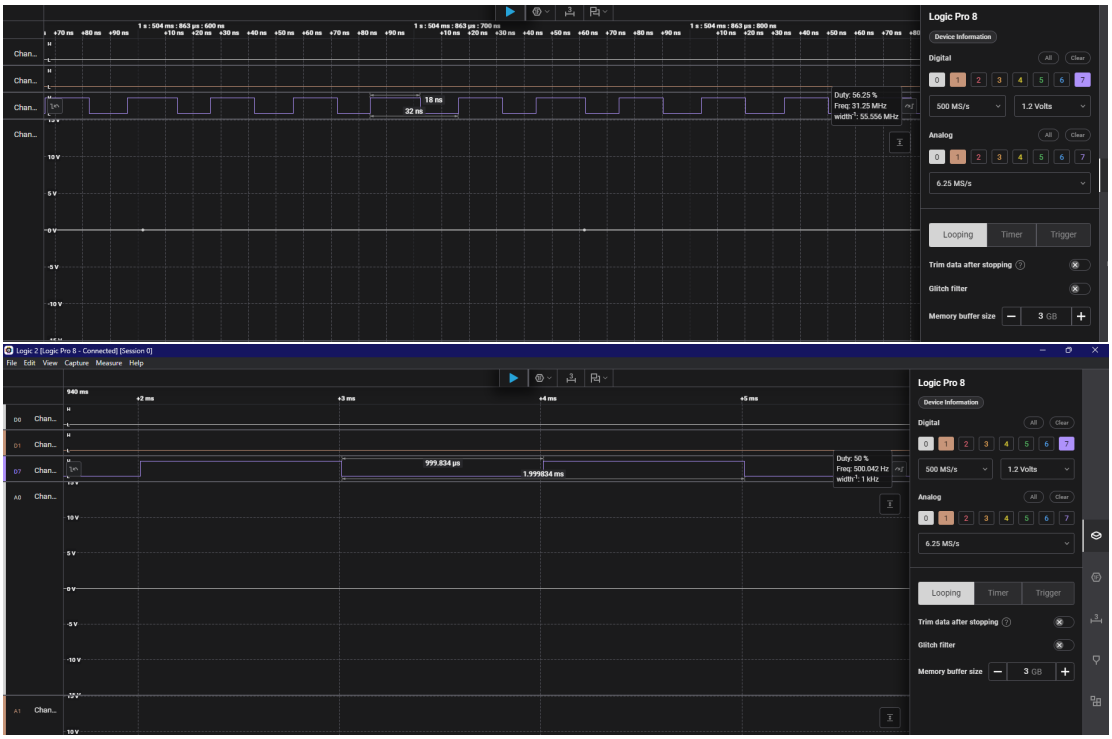


Figure 4.2: Logic analyzer measurements of the GPIO toggling task (Top view, without using a delay, bottom view with delay).

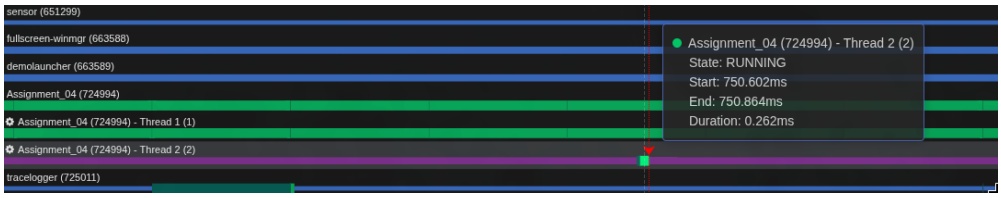


Figure 4.3: Profiling view showing the execution times of thread one. Each thread iteration took about 0.25ms to execute.

shorter periods and produced the profile shown in Figure 4.4 for the same task.

4.2.4 Task 2: CRC16 Checksum

For the checksum task, a CRC16-CCITT implementation was written to compute a checksum over a simulated 1MB flash segment, represented as a static array filled with a dummy byte pattern `0xA5`. The polynomial `0x1021` with an initial value of `0xFFFF` was used. The resulting 2-byte checksum is then encrypted using AES-128 in CTR mode via the tiny-AES library before being sent over the serial line. The task runs periodically every 500ms using `clock_nanosleep()`. The execution time of the checksum task was measured using the system profiler, as shown in Figure 4.5. The dominant cost is the CRC16 computation over the 1MB flash segment, with a WCET of approximately 60ms

Figure 4.4: Profiling view showing the execution times of thread one on the virtual target. Here each iteration takes about 0.15ms due to more powerful host hardware.

and a worst case response time of 70ms, well within the 500ms period.

4.2.5 Task 3: Diagnostic Task

The diagnostic task receives and decrypts incoming commands using AES-128 CTR mode, builds a dummy status message, encrypts it, and sends it back over the serial line. Both receive and send are implemented as dummy functions as per the assignment specification. The task runs periodically every 100ms using `clock_nanosleep()`. As shown in Figure 4.6, each iteration takes approximately 0.04ms to execute, and the response time is about 0.2ms.

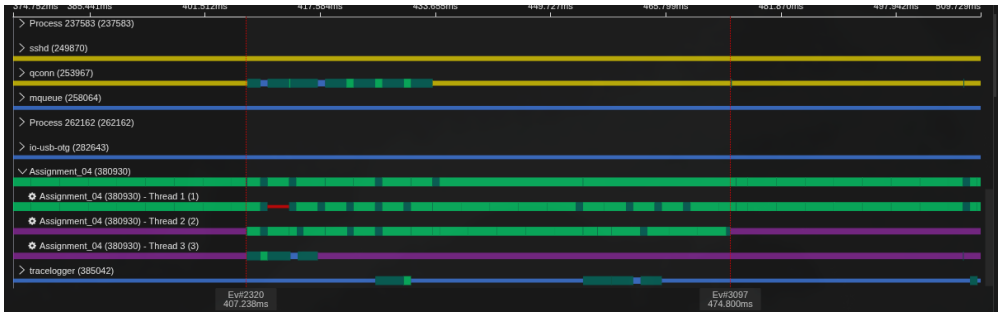


Figure 4.5: Profiling view showing the execution times of the checksum task (Thread 2). Each iteration took approximately 70ms.

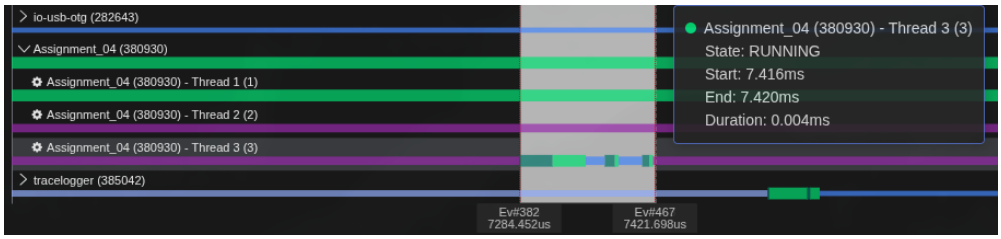


Figure 4.6: Profiling view showing the execution times of the diagnostic task (Thread 3). Each iteration took approximately 0.04ms.

4.3 Results

Running all three, now accordingly named threads at once, yields the profiling result shown in Figure 4.7. The view also shows that the threads get interrupted frequently, either by each other or by other processes on the system. This appears in the dark green sections of the timeline, where a thread is ready but not running, and the light green sections where the thread is running. Especially tasks with a long execution time, like the checksum task, are preempted quite often.

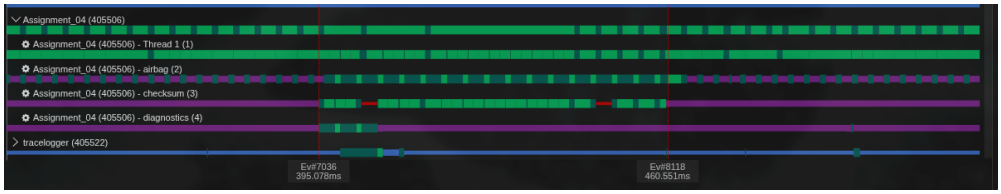


Figure 4.7: Profiling view showing the execution times of all three threads.

Observing the traces from the QEMU VM, the longest response times for each of the threads are:

- Airbag Thread: 0.2ms
- Checksum Thread: 80ms
- Diagnostic Thread: 0.30ms

The scheduling simulation in Cheddar for the same task set is shown in Figure 4.8, using a Rate Monotonic scheduling policy. It confirms that the configured capacities and periods are schedulable on one processor and illustrates the expected preemptions and context switches over the simulated interval. The second image shows the same simulation but with wrong priorities, which lead to missed deadlines for the airbag task, which is the most critical one.

4.3.1 Shared Resources and Priority Inversion

The only shared resource in the system is the serial line, accessed by both the checksum task (Task 2) and the diagnostic task (Task 3). Access is protected using a `pthread_mutex`. A bounded priority inversion can occur when Task 2 holds the mutex and is preempted by Task 3, which then blocks waiting for the mutex to be released. Since Task 2 holds

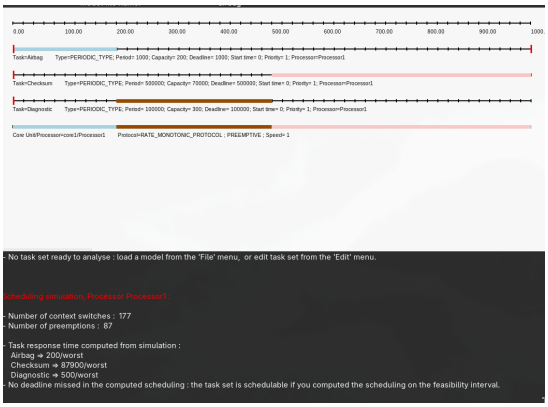


Figure 4.8: Cheddar scheduling simulation (Rate Monotonic) with correct priorities.

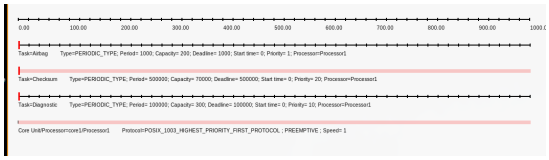


Figure 4.9: Cheddar scheduling simulation with wrong priorities.

the mutex only briefly, the inversion duration is negligible. The AES key and IV are read-only constants and require no protection.

A deadlock is impossible in this system as there is only one shared mutex. Deadlock requires a circular wait between at least two resources, which cannot form with a single mutex.

4.3.2 Alternative Resource Protection Methods

Alternative methods for protecting shared resources include:

- **Priority ceiling protocol** – raises the priority of the mutex holder to the ceiling priority, preventing priority inversion entirely.
- **Spinlocks** – busy-wait instead of blocking. Wastes CPU cycles and is unsuitable for an RTOS where other tasks need the processor.
- **Disabling interrupts** – prevents all preemption while the resource is held. Too aggressive for an RTOS as it blocks all other tasks including higher priority ones.
- **Lock-free data structures** – avoid locking entirely using atomic operations. Complex to implement correctly and overkill for a simple 2-byte serial write.

A plain mutex was chosen as it is simple, well understood, and sufficient given the short duration the resource is held and the large timing margins available.

Chapter 5

Assignment 5 — Real-time Application with QNX, HAM

Chapter 6

Assignment 6 — Real-time Application with QNX, IPC Messaging

Chapter 7

Assignment 7 — Real-time Application with QNX, Device Drivers

Chapter 8

Assignment 8 — Real-time Application with QNX, CAN Bus

f

[1] S.M. Mahmud and A.I. Alrabady. "A New Decision Making Algorithm for Airbag Control". *IEEE Transactions on Vehicular Technology* 44.3 (Aug. 1995), pp. 690–697. DOI: 10.1109/25.406638.